myAssistente

Personal Desktop Assistant – Toolkit Documentation

Release 5.X · English Manual

Content: Introduction Installation Screenshots Default Commands User and Assistant Names Fixed Replies Documentation

---

User Manual

Updated for release 5.X

Getting started guide

- **Purpose:** This manual walks you through the first installation and configuration of myAssistente.
- **Requirements:** The myAssistente executable for your operating system, or the Python files myAssistente.py and (optional to use AIML language) aiml_parser.py with Python installed on your machine (cross-platform).
- **Sections:** Installation, First launch, Try it out

> **Download** all required files from the Assistente GitLab repository.
> Italian and English are included inside Windows and Linux packages. If you need additional languages (15 ready-to-use), download the `lang_XX.json` files from the localization-file folder.
> Some AIML examples are also included; additional files in English are available in the aiml-file folder or you may use any standard AIML file available online.

Installation

The myAssistente application is portable and can be launched from any location, including a USB drive.

Windows

Download and extract `myAssistente_[last release]windows.zip`
Run `myAssistente.exe`

Linux

Download and extract `myAssistente[last release]linux.zip`
Run `myAssistente`

Python

Download `myAssistente[last release].py`
Download the optional AIML engine `aiml_parser.py`
Download the full `_dati/` folder
Ensure your Python installation includes all required dependencies (see README)
Place `myAssistente_[last release].py`, `aiml_parser.py`, and the `_dati/` folder in the same directory
Run `myAssistente_[last release].py`

All Operating Systems

Optional: add any extra language files `lang_XX.json` to the `_dati` folder (where `lang_it.json` and `lang_en.json` are located), and any AIML files to `_dati/aiml/[language]/`.

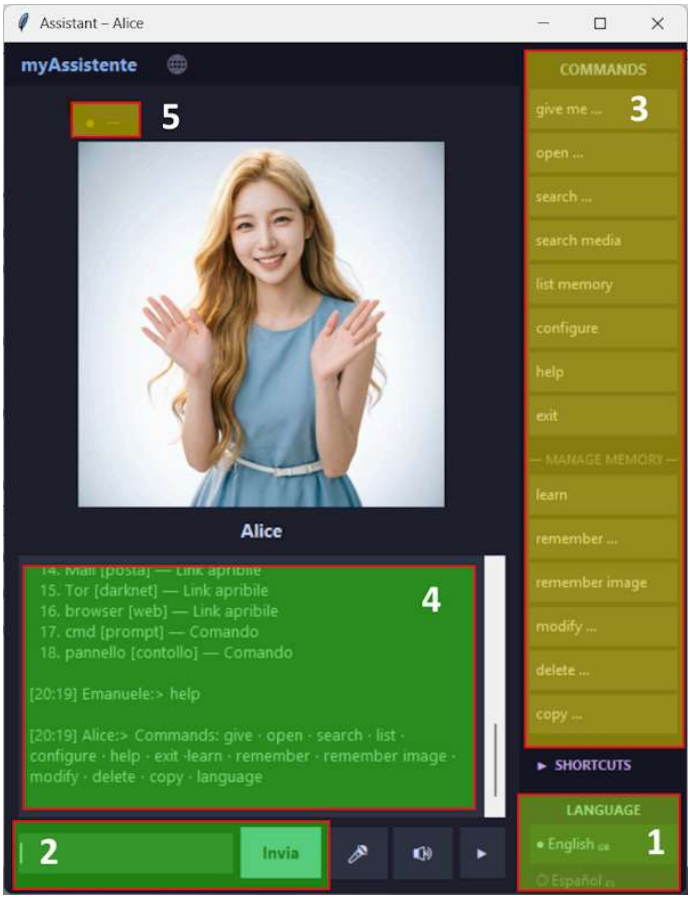
To customize the default language and other parameters, refer to the *JSON Configuration Manual*. AIML documentation is widely available online; the few optional parameters specific to myAssistente will be described in the *AIML Manual*.

File structure:

```
myAssistente    <-.py, .exe or no extension for linux
aiml_parser.py  <-.necessary for python users only
        _dati/
        config.json
        memory.json
        memory.json.bak
        langIT.json <-default language code
        langEN.json <-default language code
        langXX.json <-any other language codes
        asset/      <-your personal files will be stored here
            avatar/             <-avatar images and video
        aiml/                   <-AIML optional files
            IT/                 <-AIML files for Italian language
                saluti.aiml  <-optional aiml files
                (any number of .aiml files to be uses for Italian l
            EN/                 <-AIML files for English language
                greetings.aiml
                (any number of .aiml files to be uses for English l
            XX/ create folder with your language and insert AIML fi
```

Screenshots

Welcome Screen



1) **Language selection area**.
If the language is not correct at startup, change it using the buttons at the bottom right.
NOTE: languages found in the `_dati` folder are detected automatically. The program starts in the last selected language.

2) **Command input area**. Press the microphone to use voice commands.
Note: STT (Speech to Text) uses **Google APIs**, and audio is sent to Google. For full privacy, do not use the microphone: type commands instead.

3) **Command buttons**. If you already know the commands, hide this sidebar using the arrow at the bottom right, next to the speaker icon.

4) **Assistant response area**. Responses appear here and are spoken aloud when audio is enabled. To stop playback, type `ok` or press the “Quiet please” shortcut button (button name configurable).

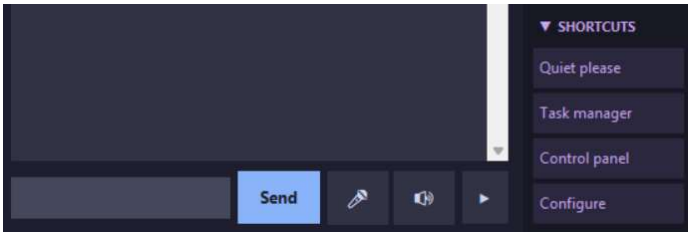
5) **Privacy indicator**
At the top left you will find the answers source indicator, helping you stay in control of your privacy.

> When the red 🔴 warning appears, your request has been sent to an external AI provider. Voice commands are always sent to Google Speech API for transcription, without indication. For 100% privacy, **do not use the microphone and do not enable external AI APIs**.

🟢 green — memory: response retrieved internally from `memory.json` (dammi, cerca, apri) — No external data sent
🔵 blue — AIML: response generated internally from `aiml/langXX/[file].aiml` files — No external data sent
🔴 red — AI: response generated by an external AI provider using your API key
⚫ grey — system messages: internal fallback responses when no command is recognized and no external AI is configured

Response search priority:
1) ⚫ grey: Internal TTS stop words (`ok, stop, quiet, silence...`) — interrupts voice output
2) ⚫ grey: Active dialog states — if a dialog is in progress (`remember, learn, delete, modify, configure...`) from system
3) 🟢 green: Fixed responses — from `config.json`
4) 🟢 green: `Commands – open, get, search, remember, learn, list, configure, exit…` from `memory.json`
5) 🔵 blue: AIML — from `.aiml` files in the current language folder `_dati/aiml/[LANG]/`
6) 🔴 red: External AI — OpenAI or Gemini (if enabled and mode allows)
7) ⚫ grey: _non_capito() — system message for unrecognized input

Shortcut Panel



Click the arrow next to SHORTCUT to open/close the customizable quick-command panel.
All buttons in this area can be customized (see the *JSON Configuration Manual*).

Default Commands

Command	Description (from lang_en.json)
`dammi... <name>`	Retrieve data from memory
`apri... <name>`	Open files, links, or programs
`cerca... <query>`	Search in memory
`cerca media <query>`	Search images, music, and videos in `_dati/asset`
`elenca memoria`	List all memory entries
`configura`	Edit config.json
`aiuto`	Show available commands
`esci`	Exit myAssistente
`--- MEMORY MANAGEMENT ---`	
`impara`	Guided entry creation
`ricorda... <name> <data>`	Save information to memory
`ricorda immagine <name> <data>`	Save images to memory

Command	Description (from lang_en.json)
`modifica... <name>`	Edit an entry
`elimina... <name>`	Delete an entry
`copia... <name>`	Duplicate an entry
`SHORTCUT` *list of quick commands*	
`LINGUA` `language name or code <code>`	Change language

> **Note** All command names can be customized in `lang_XX.json`. Each command may have multiple synonyms: the first one appears in the command panel; the others can be used as alternatives.

User and Assistant Names

Type or press the microphone and say:

```
[23:09] user:> configura breve
[23:09] Assistant:> Changing assistant name: nome_avatar current va
Enter a new name or leave empty to cancel
[23:09] user:> Alice
[23:09] Alice:> Parameter 'nome_avatar' updated to 'Alice'
[23:09] Alice:> Changing user name: nome_utente current value = 'ut
Enter a new name or leave empty to cancel
[23:09] user:> Emanuele
[23:09] Alice:> Parameter 'nome_utente' updated to 'Emanuele'
[23:09] Alice:> Configuration complete.
```

> **Note** You can perform the same configuration by editing `_dati/config.json` with any text editor (e.g., Notepad).
>
> ```
> {
>  "nome_avatar": "Assistant",   <- assistant name
>  "nome_utente": "user",        <- user name
>  "avatar_iniziale": "benvenuto",
>  "avatar_random": ["sorridente","coniglio","soddisfatto"],
> ```

Fixed Replies

You can preconfigure optional fixed replies for specific user inputs by editing the following section in `_dati/config.json`:

```
  "risposte_fisse": {
    "grazie": "Prego, {nome_utente}!",
    "ciao": "Ciao, {nome_utente}!",
    "come stai": "Sto benissimo, grazie!"
  }
}
```

The first value is the user's input; the second is the assistant's reply. You may use variables such as `{nome_utente}`. These entries are optional; used if you need to define responses that must take priority over other configuration files.

Documentation

Advanced documentation for release 5.x is **IN PROGRESS**.
Files will be available in the /DOCS folder of the repository. For missing manuals, refer to the English or Italian documentation from previous releases.

- **config.json** — configure assistant name, user name, startup language (handled automatically), STT/TTS parameters, optional AI provider APIs, and fixed replies.
- **memory.json** — where the assistant stores (impara/ricorda) and retrieves (apri/cerca/lista) information.
- **lang_XX.json** — customize all language settings.
- **aiml_XX.aiml** — standard AIML files for conversational patterns.

---

myAssistente – Personal Desktop Assistant Toolkit

Author's website · © 2026 Emanuele Cassani